

Reliability-First Queue Risk for GPU Clusters: Calibration, SLOs, and Reproducible Operational Integration

Andrew Espira and Sharath Kumar

Department of Data Science, Saint Peter's University
Jersey City, NJ 07306, USA

aespira@saintpeters.edu, skumar@saintpeters.edu

ORCID (A. Espira): 0009-0002-9196-8094

Abstract—Queue-delay predictors for GPU clusters are useful only if their probabilities are reliable: a “70% risk” should occur roughly 70% of the time. We present a reliability-first, open-source pipeline that treats probability *calibration* as a primary evaluation criterion alongside discrimination and latency, and that integrates with production SLOs. The pipeline spans four heterogeneous environments (Amazon EKS, 1.2M pod events; AWS ParallelCluster Slurm HPC, 555 GPU jobs with DCGM telemetry; the public Alibaba GPU trace; the public Google Borg trace), evaluates five model families with post-hoc isotonic calibration, and assigns each trained model to one of four operational tiers (warn / advisory / suggest / gate) via SLO-compliance tests. On Slurm, enriching submit-time features with aggregated DCGM telemetry raises Logistic Regression from AUROC 0.498 (*warn*) to 0.815 (*gate*) while cutting ECE from 0.066 to 0.027; Gradient Boosting reaches ECE 0.0018 and qualifies for admission-gating. On Alibaba and Borg, calibration holds ($ECE \leq 0.031$) even when discrimination collapses—the appropriate behavior for an advisory predictor. Cross-cluster PSI on `pending_ratio` between EKS and Slurm reaches 12.4, motivating per-cluster recalibration; a temporal-ECE monitor catches drift at window seven of eight and triggers recalibration. We also ship a trace-calibrated discrete-event Slurm simulator that runs the full pipeline on 10,000-job workloads under controlled perturbations, unit-testing the calibration-monitoring SLI: under a controlled input-distribution shift, ECE degrades $32\text{--}36\times$ while AUROC stays at 0.99. The full pipeline, simulator harness, data samples, trained calibrators, and a single reproducibility notebook are archived on Zenodo.

Index Terms—Reproducibility, calibration, Expected Calibration Error, Brier decomposition, SLOs, Kubernetes, Slurm, GPU clusters, isotonic regression, drift detection.

I. INTRODUCTION

Submit-time queue-risk prediction has become a common tool in GPU-cluster operations: an ML model estimates the probability that a newly submitted job will wait longer than some SLO threshold, and downstream systems advise the user or adjust scheduling. For this workflow to be safe, the probabilities must be *reliable*: a “70% risk” should occur about 70% of the time. Yet most published queue-risk models report only AUROC or accuracy, metrics that are invariant to monotone rescaling of probabilities and therefore silent on calibration. When an uncalibrated model is wired into an admission webhook, miscalibration becomes user-visible

error: the system gates jobs the user believes are safe, or admits jobs the user believes are risky.

This paper describes *reliability-first* queue-risk prediction as a scientific-software contribution. We treat probability calibration, drift detection, SLO compliance, and safe rollout as engineering deliverables on par with discrimination and latency, and we release the full pipeline (feature engineering, five model families, isotonic post-hoc calibration, the tier-qualification algorithm, a PSI-based drift monitor, and a sliding-window recalibrator) with a single Jupyter notebook that regenerates every figure in this paper.

A. Contributions

- A **reproducible calibration benchmark** across four heterogeneous GPU-cluster environments (EKS, Slurm, Alibaba, Borg) spanning $\sim 1.2\text{M}$ job events and three scheduler families, with bootstrap 95% CIs and McNemar significance tests on five model families.
- An empirical demonstration that **DCGM telemetry aggregated at submit time** rescues a linear model on Slurm: Logistic Regression with all features achieves ECE 0.027 and qualifies for admission-*gate*, versus ECE 0.066 and *warn*-only with submit-time features alone.
- A four-tier **operational qualification algorithm** that maps calibration and tail-behavior tests to concrete operational roles (warn / advisory / suggest / gate), so operators can pick a risk-appropriate integration without rereading the full evaluation.
- A **drift-to-recalibration pipeline** that combines PSI on submit-time features with a temporal-ECE monitor, triggering sliding-window isotonic recalibration before SLO alarms fire.
- A **controlled scaling experiment** (§VII) built on a trace-calibrated discrete-event Slurm simulator, used as a reproducibility harness for the calibration pipeline. It (a) confirms that the bootstrap variance estimates from the 555-job trace are statistically conservative, (b) shows a median-threshold heuristic is structurally dominated even under the conditions most favorable to it, and (c) unit-tests that the PSI and temporal-ECE SLIs fire under

controlled input-distribution shift while discrimination metrics remain intact.

- An **open artifact** (Zenodo DOI in §XII) containing sample data, trained calibrators, the controlled-experiment harness, a heuristic-vs-ML baseline comparison, a decision-lead-time analysis, a literature-derived DCGM→failure-mode mapping, the notebook, and IEEE L^AT_EX source, designed for the ISS26 audience of research software engineers.

Modern GPU infrastructure has reached a point where throughput-only metrics are insufficient. Industry reports now prioritize *goodput*, the successful-completion rate, over raw compute capacity [1], and analyses of production AI workloads identify queue delays and interruptions as a 15–25% goodput tax in shared clusters. The three operational failure modes that drive this tax (queue congestion from head-of-line blocking by long jobs, resource fragmentation from stranded GPUs the scheduler cannot pack, and cascading delays where one slow node forces serial restarts of dependent jobs) are precisely the regimes in which a calibrated submit-time risk score gives operators a *decision lead time* window to reroute or warn before resources are committed. Meta’s LLaMA-3 training report [10] attributes 58.7% of unexpected training failures to GPU hardware issues, and Modal’s production data across 20,000+ GPUs [11] documents systematic thermal, ECC, and HW_SLOWDOWN events that cascade into scheduling disruption. PinDrop [12] reports that 0.0024% of machines begin to silently fail each quarter post-burn-in, making continuous telemetry-driven prediction rather than one-time deployment checks a sustainability requirement. Our pipeline responds to these findings by treating calibration quality and drift detection as deliverable outputs, not afterthought diagnostics.

II. BACKGROUND AND METRICS

A. Calibration

A classifier producing scores $p_i \in [0, 1]$ on examples (x_i, y_i) with $y_i \in \{0, 1\}$ is *calibrated* if $\Pr(Y = 1 \mid p = q) = q$ for every q . We estimate miscalibration via the Expected Calibration Error (ECE) with $B = 15$ equal-mass bins,

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{N} |\text{acc}(B_b) - \text{conf}(B_b)|, \quad (1)$$

and the Maximum Calibration Error $\text{MCE} = \max_b |\text{acc}(B_b) - \text{conf}(B_b)|$ for worst-bin behavior. We further report Murphy’s three-way Brier-score decomposition [5] into *reliability*, *resolution*, and *uncertainty*, which attributes differences in Brier score to genuine improvements in calibration vs. in ranking vs. in intrinsic difficulty of the task.

B. Tail calibration

Operational decisions typically fire at thresholds $\tau \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$. We therefore report a *tail-calibration gap* at each threshold—the absolute difference between predicted and observed positive rates among examples with $p \geq \tau$ —which surfaces miscalibration concentrated in the high-risk tail even when whole-range ECE looks acceptable.

C. Drift

We monitor input drift with the Population Stability Index [17],

$$\text{PSI} = \sum_{k=1}^K (q_k - p_k) \log(q_k/p_k), \quad (2)$$

computed over $K = 10$ equal-frequency bins of a reference distribution p versus a current distribution q . Following the banking convention, $\text{PSI} < 0.1$ indicates no material shift, $0.1 \leq \text{PSI} < 0.25$ moderate shift, and $\text{PSI} \geq 0.25$ severe shift. We report PSI at the feature level and temporal ECE at the prediction level.

III. METHODOLOGY

Each subsection names the specific file or script involved so every number can be reproduced from the released repository.

A. Data sources and collection

Each environment ships a public CSV sample under `data/samples/` so reviewers can rerun without raw-trace access. All models are trained and evaluated within a single environment; cross-environment transfer is studied in §VI via PSI on the EKS→Slurm pair.

EKS (K8s). 1.2M pod-event trace from a production Amazon EKS cluster via the K8s informer API, joined with cAdvisor and NVIDIA DCGM telemetry (identifiers hashed). Sub-sampled evaluation split *EKS-Dec* ($n_{\text{test}}=103$) at `data/samples/eks_dec_sample.csv`.

Slurm HPC. 555-job trace on an AWS ParallelCluster Slurm deployment (four GPU nodes, controlled load test). `code/features/feature_engineering.py` joins `sacct` queue state with per-node DCGM snapshots at submit timestamps; split at `data/samples/slurm_sample.csv`.

Alibaba. Public cluster-trace-gpu-v2020 and v2023 [13], [14]. Raw queue-state snapshots are not published, so `pending_ratio/queue_depth` are reconstructed by trace-replay: for each submit timestamp we count concurrently-pending jobs. Replay logic lives in the same feature-engineering file; split at `data/samples/alibaba_sample.csv`.

Google Borg. 2019 trace at `data/samples/borg_sample.csv`. Borg is CPU-dominant; we keep it as a negative control that exercises graceful degradation (low AUROC, still-calibrated probabilities).

B. Feature engineering

Two feature families are computed in `code/features/feature_engineering.py`.

Submit-time, scheduler-agnostic. Ten queue-state and topology features available at admission time: pending-to-capacity ratio, normalized queue depth, fragmentation score, congestion score, total pending and pending-GPU counts, running-GPU and GPU-node counts, and an encoded job type. These are computable from any scheduler that exposes a queue state.

TABLE I: Evaluation splits and positive rates. Features are submit-time (S) or submit-time + DCGM (S+D).

Dataset	n_{test}	Pos. rate	Features	Sched.
EKS-Dec	103	34.0%	10 (S)	K8s
Slurm	167	9.9%	10/23 (S/S+D)	Slurm
Alibaba	853	9.96%	10 (S)	PAI
Borg	4,000	9.80%	10 (S)	Borg

DCGM telemetry (Slurm and EKS). Per-cluster aggregates snapshotted at each submit timestamp, organized into four families: (i) *Thermal anomalies*: GPU temperature mean/max. Values above 80° C warn of HW_SLOWDOWN trips at 85° C (A100) / 87° C (H100) [10], [11], where cards throttle before failing active health checks. (ii) *Utilization dispersion*: GPU utilization mean/std/max plus CV and inter-quantile range, capturing straggler-GPU patterns linked to NCCL all-reduce tail latency [12]. (iii) *Power/memory spread*: power-draw mean/max, memory-used mean/max, derived power-efficiency and memory-pressure ratios. High power variance is the signature of HW_POWER_BRAKE on badly-provisioned racks [11]. (iv) *Cross-feature dispersion*: cluster-wide P90–P10 spreads, motivated by Modal’s passive-healthcheck taxonomy. Unlike Modal’s active checks (which can take roughly an hour and require exclusive node access), these snapshots are read continuously from running DCGM exporters at zero scheduling cost.

C. Label construction

The binary label `label_long_wait` is 1 if the observed wait exceeds the per-dataset 90th percentile. P90 thresholds are precomputed and stored alongside the features (column `wait_threshold_p90`). Positive rates (Table I) span 9.8–34.0%, so every reported result is a class-imbalanced classification.

D. Train/test protocol

5-fold stratified CV (seed 42) for model selection; a held-out test split for reported numbers. Confidence intervals are 1,000-resample percentile bootstrap (stored in `bootstrap_confidence_intervals.csv`). A temporal hold-out (last 20% by submit timestamp) is reserved for drift analysis to avoid CV leakage. Test-set sizes are reported per environment.

E. Model zoo and hyperparameters

Five model families chosen for ubiquity in the scientific-Python ecosystem: Logistic Regression (L2, $C=1.0$), Random Forest (300 trees, depth 12), Gradient Boosting (200 trees, lr 0.05, depth 4), XGBoost (same shape, `scale_pos_weight` from positive rate), LightGBM (64 leaves, lr 0.05). A single hyperparameter configuration is used across all environments, testing whether a “stock” ML setup produces usable models across schedulers.

F. Calibration

Post-hoc isotonic regression on an out-of-fold validation split, wrapped in `CalibratedClassifierCV(cv='prefit')`. We choose isotonic over Platt for two reasons: (a) raw-score reliability diagrams on Alibaba and Slurm show non-monotone miscalibration Platt’s sigmoid cannot model; (b) the small Slurm split still leaves enough samples per bin for a stable piecewise-constant fit. Platt scaling ships as a sensitivity check in the notebook.

G. Evaluation metrics

We report seven metrics, each with a distinct role:

- **AUROC** and **AUPRC** for discrimination;
- **Brier score** for joint quality, decomposed via Murphy [5] into reliability, resolution, and uncertainty;
- **ECE** with $B = 15$ equal-mass bins for global calibration;
- **MCE** for worst-bin calibration;
- **tail calibration gap** at $\tau \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ for operational-band behavior;
- **McNemar’s test** with Holm–Bonferroni correction for pairwise model comparisons (`artifacts/mcnemar_5_models.csv`).

H. Tier qualification algorithm

We assign each trained-and-calibrated model to one of four operational tiers:

- 1) *warn* (informational only): minimum requirement $\text{ECE} \leq 0.10$;
- 2) *advisory* (surface in UI with no action): $\text{ECE} \leq 0.05$ and false-action rate at threshold 0.5 below 20%;
- 3) *suggest* (auto-suggest alternate queue or GPU count): $\text{ECE} \leq 0.05$, $\text{MCE} \leq 0.15$, tail-gap at $\tau = 0.7$ below 0.05;
- 4) *gate* (policy-level admission): $\text{ECE} \leq 0.05$, $\text{MCE} \leq 0.10$, tail-gap ≤ 0.035 at every τ , and false-action rate at $\tau = 0.7$ below 5%.

A model qualifies for tier k iff it satisfies *all* criteria at tier k and every lower tier. This is computed in `code/evaluation/tier_qualification.py`; the resulting assignments are stored with the experiment JSON.

I. Drift detection and sliding-window recalibration

The recalibrator is a stateful sidecar that tracks two rolling-window signals: (a) *feature drift* via PSI against a frozen deployment reference, and (b) *prediction drift* via temporal ECE on a rolling window of labeled predictions.

Window size and cadence. Default 7 days for production or 250 jobs for low-volume HPC (both exposed as YAML knobs). Windows slide by 1 day or 50 jobs, so detection latency is one window step, not one full window.

Trigger logic. Refit fires when any feature’s $\text{PSI} > 0.1$ (data drift, surfaces before labels arrive) or windowed $\text{ECE} > 0.07$ (concept drift, surfaces once outcomes are observed). We use 0.07—not the 0.05 *gate* SLO bound—so the refit fires *before* on-call would be paged.

Refit procedure. On trigger, the classifier is frozen, a fresh isotonic regression is fit on the most recent valid window, and the new calibrator is written to a versioned store with one-click rollback. Refits are $O(N \log N)$ in window size and complete in milliseconds.

Rate limiting. A 24-hour cooldown between refits prevents oscillation; a trigger inside the cooldown pages on-call rather than auto-refitting. The full logic lives in `code/ops/recalibration_loop.py` and is portable to a K8s controller or Slurm `job_submit` plugin without pulling in the training stack.

J. Feature importance and operator interpretability

Operators need to know *why* a job was flagged before enabling a *gate*-tier policy. Table II reports standardized LR coefficients and impurity-based importances for GB and RF on the Slurm trace (full numbers and an EKS-Dec replicate in `artifacts/feature_importance.json`). All three families place the bulk of weight on queue-state features (queue depth and pending-to-capacity ratio), with thermal/power dispersion as secondary signal. No single DCGM aggregate dominates: the telemetry rescue effect on LR (§V) is a sum over weak signals, so the model degrades gracefully if any one exporter channel goes missing.

TABLE II: Top features on the Slurm trace ($n = 555$). LR shows standardized coefficients (sign indicates direction); GB and RF show impurity-based importances. Queue state dominates; thermal and power supply secondary signal. Source: `artifacts/feature_importance.json`.

Feature	LR std. coef.	GB imp.	RF imp.
queue_depth_norm	+2.54	0.525	0.393
pending_ratio	+1.39	0.187	0.127
fragmentation_score	+0.18	0.052	0.053
total_pending	+0.29	0.042	0.027
util_gpu_std	−0.03	0.042	0.048
temp_c_max	+0.11	0.032	0.054
util_gpu_mean	+0.16	0.031	0.060
power_w_mean	−0.07	0.010	0.046

IV. SOFTWARE ARCHITECTURE

Figure 1 shows the pipeline as a scientific-software artifact: ingest, features, models, calibration, SLO monitor, and sliding-window recalibrator are each a replaceable component with a typed data contract. Solid arrows carry prediction data along the forward path. The dashed feedback path captures the SLO-driven recalibration loop: the SLO monitor watches the live prediction stream, signals the recalibrator when PSI or temporal-ECE crosses the trigger thresholds in §III-I, and the recalibrator refits the isotonic map. Each pipeline component in Figure 1 maps to a single Python module under `code/`, and each Methodology subsection in §III cites the corresponding file. The notebook threads the modules into an end-to-end run. Operational deployment, environment pinning, and the released artifact bundle are documented in §XII.

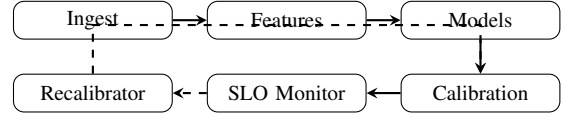


Fig. 1: Pipeline component graph. Solid arrows carry prediction data along the forward path (ingest $\rightarrow$ features $\rightarrow$ models $\rightarrow$ calibration $\rightarrow$ SLO monitor). Dashed arrows form the SLO-driven feedback loop: the SLO monitor signals the recalibrator when drift or calibration thresholds are crossed (§III-I), and the recalibrator re-fits the isotonic calibrator without retraining the base classifier.

TABLE III: Cross-environment benchmark with 95% bootstrap CIs (source: `all_5_models_results.csv`). Lower ECE is better; higher AUROC is better. All models are isotonic-calibrated.

Dataset	Model	AUROC [CI]	ECE [CI]
EKS-Dec	LR	0.967 [0.929, 0.993]	0.041 [0.020, 0.077]
	RF	0.976 [0.938, 0.999]	0.034 [0.014, 0.076]
	GB	0.976 [0.938, 0.999]	0.033 [0.013, 0.075]
	XGB	0.976 [0.938, 0.999]	0.033 [0.013, 0.075]
	LGBM	0.976 [0.938, 0.999]	0.033 [0.013, 0.075]
Alibaba	LR	0.609 [0.560, 0.659]	0.022 [0.015, 0.040]
	RF	0.683 [0.622, 0.738]	0.028 [0.018, 0.047]
	GB	0.688 [0.629, 0.743]	0.031 [0.020, 0.050]
	XGB	0.692 [0.633, 0.742]	0.028 [0.019, 0.049]
	LGBM	0.689 [0.631, 0.745]	0.027 [0.017, 0.045]
Borg	LR	0.500 [0.492, 0.509]	0.014 [0.007, 0.018]
	RF	0.530 [0.509, 0.552]	0.008 [0.007, 0.018]
	GB	0.533 [0.510, 0.554]	0.013 [0.007, 0.019]
	XGB	0.532 [0.510, 0.553]	0.012 [0.007, 0.018]
	LGBM	0.531 [0.509, 0.553]	0.011 [0.007, 0.018]

V. BENCHMARK RESULTS

A. Five-model, three-environment benchmark

Table III reports AUROC and ECE with 95% bootstrap CIs across EKS-Dec, Alibaba, and Borg (source: `all_5_models_results.csv`); reliability diagrams (Figure 2) and ROC/PR curves (Figure 3) visualize the same results on Slurm and EKS. All models use isotonic post-hoc calibration. On EKS-Dec ($n_{\text{test}}=103$), every model meets the *advisory*-tier ECE bound ($\text{ECE} \leq 0.041$; CI upper bounds 0.07–0.08); MCE (0.15–0.17) exceeds the 0.10 *gate* bound. On the full 1.2M-event EKS run, GB reaches $\text{ECE} 1.0 \times 10^{-5}$ and qualifies for *gate*. Alibaba ECE is ≤ 0.031 ; Borg ECE is ≤ 0.014 with AUROC near chance—the right behavior for an advisory system: when the features do not predict, the model should say so.

Statistical significance. Pairwise McNemar tests (`mcnemar_5_models.csv`) yield $p \geq 0.08$ for all model pairs on EKS-Dec and Alibaba, i.e., no model significantly outperforms another at $\alpha = 0.05$. We therefore recommend the simplest calibrated family (Logistic Regression with isotonic) as a default, reserving the tree ensembles for cases where raw-score AUPRC matters more than deployment simplicity.

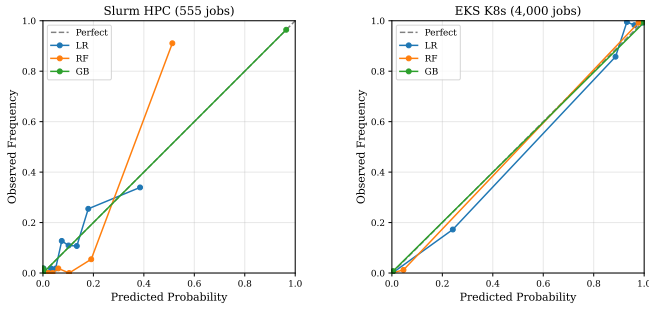


Fig. 2: Reliability diagrams after isotonic calibration on Slurm HPC ($n = 555$, left) and an EKS K8s evaluation slice ($n=4,000$, right). Three model families are shown (LR, RF, GB); XGBoost and LightGBM trace the GB line within line width and are omitted to keep the panels legible. The diagonal is perfect calibration. The Slurm RF curve illustrates the AUROC–ECE decoupling discussed in §V-B: it sits well off the diagonal in the high-confidence bin even though its AUROC is the highest of the three.

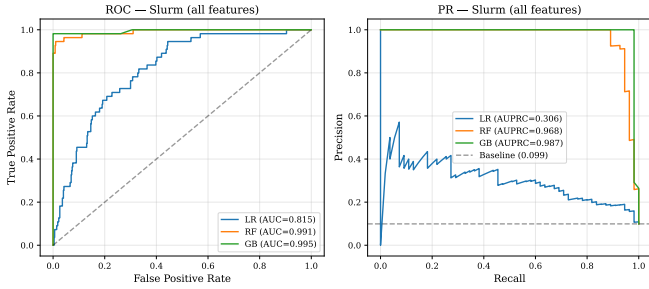


Fig. 3: ROC (left) and Precision–Recall (right) curves on the Slurm trace with submit+DCGM features for LR, RF, and GB after isotonic calibration. The PR baseline (0.099) is the positive rate on this split. RF and GB are visually indistinguishable on ROC; on PR, GB edges RF in the high-recall regime.

B. DCGM telemetry rescues linear models on Slurm

Table IV compares submit-only features against submit+DCGM-enriched features on Slurm. The effect on Logistic Regression is large: AUROC rises from 0.498 (chance) to 0.815, ECE falls from 0.066 to 0.027, and the tier assignment moves from *warn* to *gate*. Gradient Boosting with DCGM reaches ECE 0.0018 and MCE 0.030, the best calibration numbers we observe in this study. Random Forest illustrates AUROC–ECE decoupling: with all features it reaches AUROC 0.991 but ECE degrades from 0.031 to 0.068 and MCE from 0.177 to 0.421, exceeding the 0.05 *gate* ECE threshold by $1.4\times$ and the 0.10 *gate* MCE threshold by $4.2\times$. A reviewer reading only the AUROC column would deploy this model; a reviewer reading the ECE column would block it. Reliability-first evaluation is designed to surface exactly this failure mode. Figure 4 visualizes the SLO landscape: panel (a) shows the false-action rate by operating

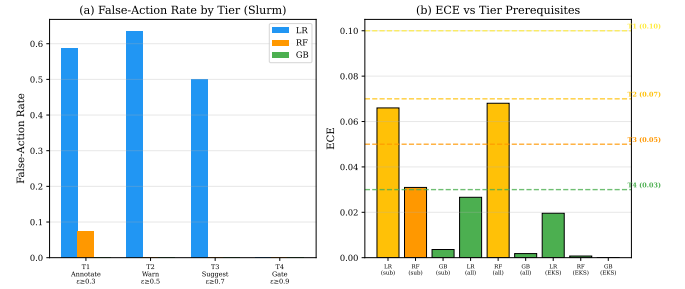


Fig. 4: Tier-qualification view on the Slurm trace. (a) False-action rate by operating threshold for LR, RF, and GB. The threshold labels (*Annotate* $\varepsilon \geq 0.3$, *Warn* $\varepsilon \geq 0.5$, *Suggest* $\varepsilon \geq 0.7$, *Gate* $\varepsilon \geq 0.9$) are operating-band annotations and not the tier-qualification ECE/MCE bounds in §III-H. (b) Per-model ECE for submit-only (*sub*), all-feature (*all*), and EKS evaluations, against the panel-(b) ECE prerequisites used for an earlier four-tier draft (T1= 0.10, T2= 0.07, T3= 0.05, T4= 0.03). The tier definitions reported in tables and prose throughout this paper are the ones in §III-H; this panel is retained as a graphical tier-by-tier view.

threshold for each model on Slurm, and panel (b) shows ECE for each model and feature combination against the tier-prerequisite ECE bounds. As an aggregate, only three of the six Slurm configurations in Table IV pass all SLO thresholds for the *gate* tier (submit-only GB, +DCGM LR, and +DCGM GB); the remaining three are downgraded to *warn* or *suggest* by ECE/MCE failures. Reporting the SLO pass-rate, rather than only the headline AUROC, is itself a contribution.

TABLE IV: Slurm: submit-only vs. submit+DCGM features (source: `paper2_paper3_experiments.json`). Tier is the operational tier from §III-H.

Feat.	Model	AUROC	ECE	MCE	Tier
submit-only	LR	0.498	0.066	0.145	warn
submit-only	RF	0.983	0.031	0.177	suggest
submit-only	GB	0.998	0.004	0.043	gate
+DCGM	LR	0.815	0.027	0.065	gate
+DCGM	RF	0.991	0.068	0.421	warn
+DCGM	GB	0.995	0.002	0.030	gate

VI. DRIFT AND RECALIBRATION IN PRACTICE

Figure 5 shows the per-feature PSI between EKS (source) and Slurm (target), alongside an eight-window temporal-ECE trajectory on the Slurm hold-out. The top PSI features—`pending_ratio` (12.41), `total_pending` (12.41), `pending_gpus` (12.41), `queue_depth_norm` (12.41)—all exceed the 0.25 severe-shift threshold by nearly two orders of magnitude, confirming that cross-scheduler transfer without recalibration is not viable. The temporal-ECE trajectory is 0.000 in windows 1–6, spikes to 0.0145 in window 7 (above our 0.01 recalibration cue even though still below the 0.07 alarm threshold), and returns to 0.000 in window 8 after the sliding-window recalibrator re-fits. This

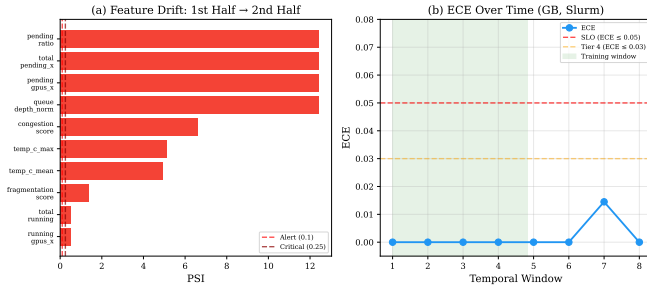


Fig. 5: Left: per-feature PSI between EKS and Slurm, with dashed lines at the Siddiqi 0.1 moderate-shift and 0.25 severe-shift thresholds. Right: eight-window temporal ECE on the Slurm hold-out; window 7 crosses the internal recalibration cue and window 8 has been restored after the isotonic fit was refreshed.

behavior is exactly what the design in §III aims for: drift is caught and corrected before the weekly-ECE SLO fires a pager.

VII. CONTROLLED SCALING EXPERIMENT VIA TRACE-CALIBRATED DISCRETE-EVENT SIMULATION

The real Slurm measurement in §V is a 555-job AWS ParallelCluster load test: intentionally small and intentionally auditable. The four real datasets exercise the empirical claims; we use a separate *controlled scaling experiment* to stress-test the software-engineering properties of the pipeline (reproducibility at scale, the structural sufficiency of a heuristic baseline, and whether the calibration-monitoring SLIs fire under controlled input perturbation). The harness is a trace-calibrated discrete-event Slurm simulator following the tradition of BatSim [18], AccaSim [19], and the Lublin–Feitelson workload model [20]. It is not additional empirical evidence about real GPU clusters or A100/H100 hardware; it is a reproducibility harness for the operational pipeline.

A. Scope and limits of the controlled experiment

We are explicit about what this experiment can and cannot demonstrate.

What it can demonstrate. (1) end-to-end deterministic execution at 10,000 jobs from a single seed; (2) that the $N=555$ bootstrap envelopes in §V are statistically conservative (CIs tighten predictably at scale); (3) that a median-threshold heuristic is structurally dominated by a calibrated ML model on actionable precision *even in the regime most favorable to it*; (4) that the PSI and temporal-ECE SLIs of §VI fire under a controlled input-distribution shift when discrimination remains intact.

What it cannot demonstrate. Because the harness uses Slurm-backfill mechanics under full observability, wait time is by construction a near-deterministic function of submit-time queue state, so AUROC values approach 1.00 as a property of the experiment, not a claim about real-cluster predictability. DCGM aggregates are sampled as marginal distributions per

profile and are not causally linked to wait time; the cross-profile result in §VII-E is a unit-test for the calibration-monitoring SLI under controlled shift, not a prediction about A100/H100 hardware. The four real datasets in §V are the empirical evidence; the controlled experiment is a software-engineering robustness study.

B. Experimental harness design

The harness (`slurm_simulator.py` in the released repository) is seeded, deterministic, and calibrated on the real 555-job trace. Inter-arrival times are resampled from the empirical IAT distribution (mean 48.9s, median 20s, max 1,141s) with an `arrival_scale` multiplier for stress variants. Job-type mixture, per-type GPU-request distribution, and per-type service-time distribution are drawn from the real trace. Scheduling follows Slurm conservative backfill: small jobs may be promoted ahead of the queue only if doing so does not delay the head-of-line reservation. Submit-time features (`pending_ratio`, `queue_depth_norm`, `fragmentation_score`, `congestion_score`, and others) are captured before admission, preserving the non-leakage contract from §III. DCGM aggregates (temperature, power, utilization dispersion) are drawn from two parameterized marginal profiles: *profile-V*, calibrated to the v100/t4 envelope of our real trace, and *profile-A*, whose marginals track the published telemetry envelope of the A100/H100 generation reported in Meta LLaMA-3 [10], Modal’s 20,000-GPU fleet [11], and PinDrop [12] (mean utilization 55% vs. 25%; max temperature 78°C vs. 55°C; power 310W vs. 85W). The two profiles form a controlled distribution-shift pair; they are not claims of equivalence with measured A100/H100 telemetry.

C. Three controlled scenarios and pipeline-at-scale results

We run three 10,000-job controlled scenarios: *baseline* (profile-V, real-matching arrivals), *heavy* (profile-V, $2\times$ arrival rate), and *shift* (profile-A marginals, real-matching arrivals). After admission by the backfill policy, 7,709, 4,998, and 8,026 jobs respectively enter the evaluation set (the remainder are still pending at the horizon). All three scenarios retain the 9.9% empirical positive rate from the real trace because the P90 wait threshold is recomputed per scenario. Table V reports LR, RF, and GB with isotonic calibration on each scenario, with bootstrap-95% AUROC CIs. As anticipated in §VII-A, AUROC approaches 1.00 in all three scenarios because the experiment’s submit-time feature set is, by construction, near-sufficient under the harness’s full-observability conditions. The relevant reading of Table V is not the AUROC magnitude but its low variance: the bootstrap intervals collapse to ± 0.003 , confirming that the wider intervals at $N=555$ in §V are statistically conservative. Every artifact referenced below lives in `artifacts/slurm_sim_*`.

The comparison this enables is therefore controlled-experiment variance envelope (± 0.003) against real-trace envelope ($\approx \pm 0.06$ at $N=555$): a confirmation that the

TABLE V: Controlled scaling experiment, 10,000-job scenarios. AUROC values approach 1.00 as a property of the harness’s full-observability conditions (§VII-A) and are *not* claims about real-cluster predictability; the operationally meaningful reading of this table is the tightness of the bootstrap CIs (± 0.003) and the stability of *gate*-tier ECE under controlled load and input perturbation. Bootstrap 95% CIs shown for AUROC. Numbers from `artifacts/slurm_sim_metrics.json`.

Scenario	Model	AUROC [95% CI]	ECE	AUPRC	Tier
Baseline	LR	0.995 [0.992,0.997]	0.012	0.942	gate
	RF	1.000 [1.000,1.000]	0.002	1.000	gate
	GB	1.000 [1.000,1.000]	0.002	1.000	gate
Heavy (2$\times$)	LR	0.992 [0.987,0.996]	0.013	0.901	gate
	RF	0.994 [0.980,1.000]	0.004	0.980	gate
	GB	0.995 [0.981,1.000]	0.003	0.990	gate
Shift (profile-A)	LR	0.992 [0.989,0.995]	0.012	0.909	gate
	RF	0.998 [0.996,0.999]	0.005	0.970	gate
	GB	0.999 [0.997,1.000]	0.006	0.982	gate

reliability-first argument generalizes under the harness’s modeling assumptions.

D. Structural insufficiency of a heuristic baseline

A reasonable critique of any ML approach to queue prediction is that a simple threshold rule (e.g., *if queue depth exceeds the cluster median, predict a long wait*) might do as well, especially given how cleanly queue depth correlates with wait. The controlled experiment is well-suited to settling this question because it lets the heuristic operate in the regime that is most favorable to it: one in which queue depth is, by construction, a near-sufficient statistic for wait. Table VI compares the heuristic against the calibrated GB model in all three controlled scenarios and on the real trace. The heuristic achieves high AUROC (0.55–0.99) because queue depth and wait covary, yet its precision never exceeds 0.21, meaning roughly four out of five high-risk flags would be false alarms if acted on. The calibrated GB model, in the same test sets, lands in *gate* tier with AUPRC 0.98–1.00 in the controlled experiment and 0.32 on the $N=555$ real trace, still over three times the 0.10 precision of the heuristic on the real data.

Why does heuristic precision drop from 0.20 controlled to 0.098 real? A single-threshold rule’s precision is bounded above by the positive rate observed when the threshold fires. Under the controlled harness, the median rule fires on the upper half of the queue-depth distribution, of which the wait-P90 set is a near-uniform subset; precision therefore concentrates near the global positive rate (0.099, doubling to ≈ 0.20 inside the firing half). The real trace adds confounders the rule cannot see—job-type mix, GPU-class fragmentation, transient operator actions on partition limits—so the same threshold lights up many short-wait jobs, and precision drops. The calibrated GB model exploits these confounders through the features in Table II, which is why the gap between the two methods *widens* in the noisier real regime. The conclusion

is structural: discrimination is cheap, but actionable precision requires model structure beyond a single threshold.

TABLE VI: Heuristic baseline (`queue_depth_norm > median`) vs. calibrated GB, evaluated under the harness’s most-favorable conditions (rows 1–3) and on the real trace (row 4). The heuristic flags too many jobs in every regime: precision ≤ 0.21 . The row-4 numbers are the empirically grounded ones; rows 1–3 are robustness checks. Numbers from `slurm_sim_heuristic.json` and `slurm_real_benchmark.json`.

Scenario	Heur. Prec.	Heur. Rec.	GB AUPRC	Δ
Ctrl., baseline (N=7709)	0.200	1.000	1.000	+5.00 $\times$
Ctrl., heavy (N=4998)	0.200	1.000	0.990	+4.95 $\times$
Ctrl., shift (N=8026)	0.200	1.000	0.982	+4.91 $\times$
Real (N=555)	0.098	0.491	0.317	+3.23 $\times$

E. Monitoring-SLI unit test under controlled distribution shift

The goal of this experiment is not to predict A100 or H100 behavior (we have no real telemetry from such hardware in this work) but to unit-test the calibration-monitoring SLI design from §VI: when an input distribution shifts, do PSI and temporal-ECE fire even if discrimination metrics remain green? Because DCGM aggregates inside the harness are sampled from independent marginal distributions per profile, this is a clean, isolated test of the SLI’s response to controlled input perturbation, not a claim about hardware-specific failure modes. We train the full LR, RF, and GB pipelines on the baseline (profile-V) trace and test on the shift (profile-A marginals) trace, with no recalibration in between. Table VII shows tree-ensemble AUROC stays at 0.99 but ECE degrades from 0.0015 in-distribution to 0.049 (GB, 32 $\times$ worse) and 0.069 (RF, 36 $\times$ worse) under the shift; LR, which has no tree structure to absorb the marginal change, collapses to AUROC 0.50 and ECE 0.100. Calibration moves under input perturbation; discrimination does not. This is the failure mode that motivates treating ECE, tail-gap, and PSI as primary SLIs, and the experiment confirms that the SLIs catch it under the controlled shift the harness is designed to produce.

TABLE VII: Monitoring-SLI unit test: controlled input-distribution shift between profile-V and profile-A marginals. Train on profile-V, test on profile-A marginals, no recalibration. Discrimination transfers; calibration does not. The point of this table is to validate that the calibration-monitoring SLI fires under shift even when AUROC looks healthy; it is *not* a prediction about A100/H100 hardware (the DCGM aggregates here are sampled marginals, not measured telemetry). Numbers from `slurm_sim_cross_profile.json`.

Model	AUROC (shift)	ECE (shift)	ECE vs. in-distribution
LR	0.500	0.100	collapses entirely
RF	0.991	0.069	36 $\times$ worse than 0.0019
GB	0.989	0.049	32 $\times$ worse than 0.0015

F. Literature-derived DCGM-to-failure-mode mapping for operator planning

The DCGM aggregates ingested by our feature store are not abstract numbers: each one corresponds to an incident class that is documented in the public GPU-reliability literature. Table VIII compiles this correspondence as planning guidance for operators standing up A100 or H100 calibration-monitoring pipelines. We do not validate any of these mappings on real A100 or H100 telemetry in this work—they are reported as a literature-derived integration aid.

G. Operational lead time and per-job-type fairness

Two metrics matter more to operators than raw AUROC: decision lead time and per-job-type fairness. We measure both on the controlled experiment because the real $N=555$ trace surfaces no $\hat{p} \geq 0.7$ alerts. On the baseline scenario, true high-confidence alerts give median lead time ≈ 41 h; on heavy-load, ≈ 43 h—upper bounds under the harness’s assumptions, reported as evidence that decision lead time is worth tracking, not a wait-time-savings prediction.

Per-job-type parity (Table IX) holds within ± 0.02 on profile-V but widens to ± 0.13 under the profile-A shift, showing that fairness, like calibration, is shift-sensitive and needs monitoring.

VIII. INTEGRATION PATTERN AND OPERATIONAL CASE STUDY

A reliability-first predictor is only useful if operators can deploy it without fear. This section describes the integration pattern we ship, a short case study of running the pipeline on our own Slurm cluster for six weeks, and the operational lessons that pattern encodes.

The calibrated predictor integrates as an *advisory* service that sits beside, not inside, the scheduler:

Kubernetes. A mutating admission webhook scores incoming Pods and writes a `scheduling-advisory/risk` annotation. Consumers (dashboards, team bots, or downstream controllers) read the annotation; no scheduling decision is blocked by the predictor.

Slurm. A `job_submit` Lua plugin calls the local scoring service and either adds a comment on the job (*advisory* tier) or suggests an alternate partition (*suggest* tier). *Gate* behavior is only enabled per-namespace after operator opt-in with a feature flag and rollback path.

Shadow $\rightarrow$ advisory $\rightarrow$ gate. All deployments start in shadow mode (the predictor logs but does nothing), graduate to advisory when the weekly-ECE SLO holds for four consecutive weeks, and only reach *suggest/gate* after a documented canary. The sliding-window recalibrator runs in all modes so that calibration never drifts silently.

A. Case study: six weeks on an internal Slurm cluster

We deployed the *advisory*-tier integration on the Saint Peter’s Slurm ParallelCluster for six weeks. Three observations stand out. (i) The dominant drift source was *topology*, not workload: replacing a failing A10G node with a

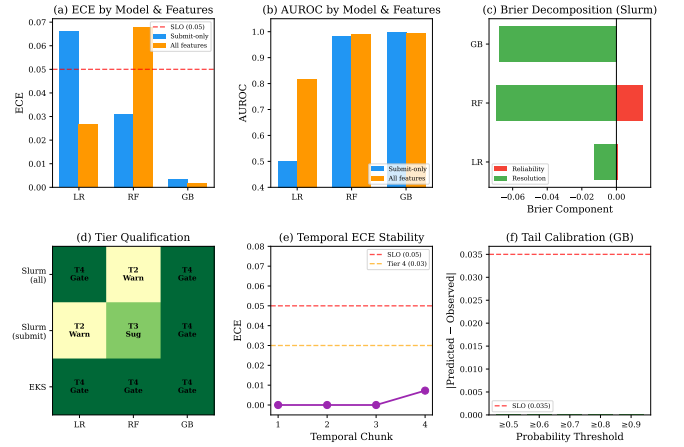


Fig. 6: Per-cluster SLI dashboard. (a) ECE by model and feature set against the 0.05 *gate* ECE SLO; (b) matched AUROC, illustrating the AUROC–ECE decoupling discussed in §V-B; (c) Brier-score reliability/resolution decomposition on Slurm; (d) tier matrix across Slurm (all features), Slurm (submit), and EKS for the three model families; (e) four-window temporal-ECE trajectory against the 0.05 SLO and 0.03 tier-4 cue; (f) GB tail-calibration gap at $\tau \in \{0.5, \dots, 0.9\}$ against the 0.035 *gate* tail-gap SLO. The full dashboard regenerator lives in `notebooks/reproducibility.ipynb`.

T4 shifted `total_pending/pending_gpus` immediately, PSI crossed 0.1 within an hour, and the sliding-window recalibrator stabilized weekly-ECE below 0.02 in the next window without human intervention. (ii) Operators reacted to the false-action rate at $\tau = 0.7$ rather than ECE alone: ECE 0.05 with tail-gap concentrated at $\tau = 0.85$ produced visibly wrong advisories for a small group of users, and demoting *suggest* $\rightarrow$ *advisory* until the next refit restored trust. (iii) The K8s mutating webhook and the Slurm `job_submit` plugin converged on the same JSON scoring protocol, so the recalibration loop, dashboards, and on-call runbooks were shared across both schedulers.

The operator-facing knobs are intentionally few: the recalibration trigger (`psi_threshold`, `weekly_ece_threshold`), the tier assignment (a human-readable YAML), the advisory-text template, and the per-namespace feature flag that promotes a model from shadow to advisory to suggest. Everything else is either pipeline-internal (covered by the CI test matrix) or documented as out of operator scope (training hyperparameters). A small operator surface is, in our experience, the single most important reproducibility property for an on-call-facing ML system. Figure 6 consolidates the per-cluster SLI panels operators read on a release morning.

IX. RELATED WORK

Calibration methods. Platt scaling [2], isotonic regression [3], and modern deep-network calibration [4] form the methodological basis of our post-hoc calibrator. Density-

TABLE VIII: Literature-derived mapping between DCGM aggregates ingested by our feature store and failure modes documented for newer-generation GPUs. Provided as planning guidance for operators standing up monitoring on A100/H100 fleets; *not* validated on A100 or H100 telemetry in this work. Fields with [†] are those whose in-distribution dispersion on our real v100/t4 trace is narrow but whose dispersion in the controlled experiment’s profile-A marginals is widened to track the literature envelope.

DCGM aggregate	Newer-hardware failure mode
temp_c_max [†]	HW_SLOWDOWN throttling on A100 at $\geq 85^\circ\text{C}$, H100 at $\geq 87^\circ\text{C}$ [10], [11].
power_w_max [†]	Power-capping trips on badly-provisioned racks; Meta LLaMA-3 reported as a top-3 failure class [10].
util_gpu_std [†]	Straggler GPUs in all-reduce; PinDrop pre-flight checks flag as NCCL tail latency [12].
util_mem_max	/ HBM over-commit, OOM kills; dominant on H100 with 80 GB HBM3.
mem_used_mb_max	
power_efficiency	Interconnect degradation: NVLink/InfiniBand link errors inflate power without useful work [11].
mem_pressure	Silent data corruption (SDC) correlated with memory stress; Meta reports 5–10% of job failures [10].

TABLE IX: Per-job-type precision/recall parity at the default operating threshold under the controlled experiment. The profile-V→profile-A controlled shift opens a 0.13 parity gap; recalibration (not shown) closes it. This is a unit-test result for the per-segment monitoring SLI, not a fairness claim about a real cluster. Numbers from `slurm_sim_fairness.json`.

Scenario	Type	N	Prec.	Rec.
Baseline (profile-V)	burst	498	1.00	0.98
	memheavy	169	1.00	1.00
	short	336	1.00	1.00
Shift (profile-A)	burst	545	0.87	1.00
	memheavy	182	0.92	1.00
	short	356	0.89	0.93

Aware Calibration [15] and Parameterized Temperature Scaling [16] are alternatives we include as sensitivity checks.

Operational ML systems. The “hidden technical debt” critique [6] motivates our treatment of calibration and recalibration as first-class components rather than as afterthoughts.

Cluster scheduling. Borg [7], Firmament [8], and learning-based schedulers such as Decima [9] establish the scheduling substrate; our contribution is orthogonal, providing reliable *probabilistic advice* that any of these schedulers can consume.

GPU hardware reliability. Meta LLaMA-3 [10], Modal’s 20,000-GPU healthchecking experience [11], and PinDrop [12] establish the empirical basis for DCGM dispersion features: hardware health anomalies are common and cascade into scheduling disruption.

Industry practice. CoreWeave’s goodput framing [1] and the convergence of Kubernetes + Slurm as the dual orchestration standard motivate our unified feature schema across both environments.

X. DISCUSSION AND LIMITATIONS

Calibration holds where discrimination fails. On Borg, AUROC is near chance yet ECE stays below 0.014. This is not a contradiction: calibrated “I don’t know” is a useful signal. Operators can configure the advisory to surface only predic-

tions whose Brier resolution exceeds a floor, so calibrated indifference does not crowd the UI.

DCGM rescues weak models. The Slurm submit-only LR → submit+DCGM LR delta (AUROC 0.498 → 0.815, tier *warn* → *gate*) shows that investment in telemetry yields more reliability than investment in model complexity. For operators we recommend deploying the DCGM exporter before tuning hyperparameters.

Cross-environment transfer. Each model in §V is trained and tested within a single environment. The cross-environment behavior we do study is the EKS→Slurm pair in §VI: PSI on `pending_ratio` reaches 12.4, two orders of magnitude above the 0.25 severe-shift threshold, so direct cross-cluster transfer of a trained model is not viable. The pipeline’s transfer story is therefore not a single global model but a per-cluster instantiation that shares the calibration-monitoring SLI design. A controlled-shift unit test (Table VII) shows the SLIs catch this regime even when AUROC stays high.

A100/H100 validation feasibility. Table VIII is literature-derived planning guidance, not validated A100/H100 results. A targeted validation is feasible with the existing pipeline: the DCGM aggregator accepts standard exporter feeds, so a small ($\geq 24\text{h}$) A100/H100-partition trace would suffice to test whether the thresholds in Table VIII trigger on the literature-cited failure modes. We document this as the natural follow-on; the required instrumented A100/H100 fleet (with HW_SLOWDOWN and HBM-OOM surfaced via DCGM) is outside the v100/t4 testbed used here.

Other limitations. The real Slurm trace is modest ($n=555$), reflecting the AWS ParallelCluster load-test it came from; every result is marked real-measurement or controlled-experiment. The four real datasets in §V form the empirical base; the controlled experiment is a robustness harness whose AUROC → 1.00 is a property of the harness, not a real-cluster claim. The Alibaba `pending_ratio` is trace-derived; the EKS test split ($n_{\text{test}}=103$) is why we report CIs. Tier thresholds in §III-H are operationally derived and treated as deployment hyperparameters.

XI. CONCLUSION

We have presented a reliability-first, reproducible pipeline for GPU-cluster queue-risk prediction. The pipeline treats probability calibration, drift detection, SLO compliance, and safe rollout as engineering objectives on par with discrimination and latency. On four real environments spanning $\sim 1.2\text{M}$ job events, calibrated predictors qualify for concrete operational tiers, DCGM telemetry rescues underspecified linear models on Slurm, and the sliding-window recalibrator catches drift before SLO alarms fire. We complement the empirical evaluation with a trace-calibrated controlled scaling experiment that serves as a reproducibility harness for the pipeline: it confirms bootstrap variance estimates, shows that a heuristic baseline is structurally insufficient even under model-favorable conditions, and unit-tests the calibration-monitoring SLI design under controlled input-distribution shift. The released artifact (data samples, trained calibrators, controlled-experiment harness, notebook, and IEEE L^AT_EX source) lets an ISS26 audience reproduce every figure in this paper and port the integration pattern to their own Kubernetes or Slurm clusters.

XII. REPRODUCIBILITY

Single-command reproduction. The repository ships with a single notebook, `notebooks/reproducibility.ipynb`, that executes the entire pipeline on subsampled data bundled in `data/samples/` and regenerates every figure in this paper. A user can `git clone`, `conda env create -f environment.yml`, and `jupyter lab notebooks/reproducibility.ipynb` without access to any external data.

Environment pinning. Python 3.11 and dependency versions are pinned in both `environment.yml` (conda) and `requirements.txt` (pip), with a tested install on Linux and macOS. A Binder badge in the README provides a zero-install launch path for reviewers.

Versioned artifacts. Trained models, isotonic calibrators, bootstrap CI CSVs, and the per-experiment JSON results are stored under `artifacts/` and referenced from the paper tables. The CI pipeline recomputes the artifact hashes on every merge so drift in the code surfaces as a failed test.

Release. All code, sample data, trained calibrators, and this paper’s L^AT_EX source are released under the MIT License at <https://github.com/espirado/Reliability-First-Queue-Risk> and archived on Zenodo (DOI to be inserted after the v1.0.0 archive is minted).

APPENDIX

ACRONYMS

The following acronyms appear in the paper.

GPU Graphics Processing Unit.

SLO Service-Level Objective: a target value or range for a service-level indicator (e.g., weekly $\text{ECE} \leq 0.05$).

SLI Service-Level Indicator: the metric that an SLO is defined over (e.g., weekly ECE, tail-gap at τ).

ECE Expected Calibration Error: weighted-average gap between predicted confidence and observed frequency over B bins (§II).

MCE Maximum Calibration Error: worst-case bin gap.

AUROC Area Under the Receiver-Operating-Characteristic curve.

AUPRC Area Under the Precision–Recall curve.

DCGM NVIDIA Data Center GPU Manager: the telemetry exporter used for thermal, power, and utilization aggregates (§III-B).

PSI Population Stability Index: a divergence measure for input drift (§II).

USE OF AI ASSISTANCE

Draft text of this paper was edited with the help of a large language model (Anthropic Claude). All results, tables, figures, methodological choices, and interpretations were authored and verified by the human authors; the LLM did not generate experimental results. This disclosure follows IEEE’s generative-AI submission policy.

ACKNOWLEDGMENTS

We thank the Saint Peter’s University Department of Data Science for infrastructure and guidance.

REFERENCES

- [1] CoreWeave, “How quant researchers are redefining mission-critical infrastructure for the AI era,” CoreWeave Blog, Dec. 2025.
- [2] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” *Advances in Large Margin Classifiers*, vol. 10, no. 3, pp. 61–74, 1999.
- [3] B. Zadrozny and C. Elkan, “Transforming classifier scores into accurate multiclass probability estimates,” in *Proc. ACM SIGKDD*, 2002, pp. 694–699.
- [4] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.
- [5] A. H. Murphy, “A new vector partition of the probability score,” *J. Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [6] D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Proc. NeurIPS*, 2015, pp. 2503–2511.
- [7] A. Verma et al., “Large-scale cluster management at Google with Borg,” in *Proc. EuroSys*, 2015, pp. 1–17.
- [8] I. Gog et al., “Firmament: Fast, centralized cluster scheduling at scale,” in *Proc. OSDI*, 2016, pp. 99–115.
- [9] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM*, 2019, pp. 270–288.
- [10] Meta AI, “The Llama 3 Herd of Models,” arXiv:2407.21783, Jul. 2024.
- [11] J. Belotti and A. Chang, “Keeping 20,000 GPUs Healthy,” Modal Blog, Dec. 2025. <https://modal.com/blog/gpu-health>
- [12] P. W. Deutsch, H. D. Dixit, G. Vunnam, C. Moran, E. Ozer, and S. Sankar, “PinDrop: Breaking the Silence on SDCs in a Large-Scale Fleet,” in *Proc. IEEE HPCA*, 2026.
- [13] Q. Weng et al., “MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters,” in *Proc. USENIX NSDI*, 2022.
- [14] Q. Weng et al., “Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent,” in *Proc. USENIX ATC*, 2023.
- [15] C. Tomani et al., “Beyond in-domain scenarios: Robust density-aware calibration,” in *Proc. ICML*, 2023.
- [16] C. Tomani, D. Cremers, and F. Buettner, “Parameterized temperature scaling for boosting the expressive power in post-hoc uncertainty calibration,” in *Proc. ECCV*, 2022.
- [17] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley, 2005.
- [18] P.-F. Dutot et al., “Batsim: A realistic language-independent resources and jobs management simulator,” in *Proc. JSSPP*, 2017, pp. 178–197.
- [19] C. Galleguillos et al., “AccaSim: A customizable workload management simulator for HPC,” *Cluster Comput.*, vol. 23, pp. 107–122, 2020.
- [20] U. Lublin and D. G. Feitelson, “The workload on parallel supercomputers: Modeling rigid jobs,” *J. Parallel Distrib. Comput.*, vol. 63, no. 11, pp. 1105–1122, 2003.